# EXPERIMENT 18

High Level Programming Extensions – Cursors

## Aim

To use a MySQL cursor inside a stored procedure to read employee names one by one and display them.

## Software Required

MySQL Server and MySQL Workbench or MySQL command-line client.

## SQL Code

```
CREATE DATABASE IF NOT EXISTS CompanyCursorDB;
USE CompanyCursorDB;

DROP TABLE IF EXISTS Employee;
CREATE TABLE Employee (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   department VARCHAR(30)
);

INSERT INTO Employee VALUES
(301, 'Divya', 'IT'),
(302, 'Rahul', 'HR'),
(303, 'Sita', 'Sales');

DROP PROCEDURE IF EXISTS DisplayEmployeeNames;
DELIMITER //
CREATE PROCEDURE DisplayEmployeeNames()
BEGIN
   DECLARE finished INT DEFAULT 0;
   DECLARE employee_name VARCHAR(50);

   DECLARE emp_cursor CURSOR FOR
      SELECT emp_name FROM Employee ORDER BY emp_id;

   DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;

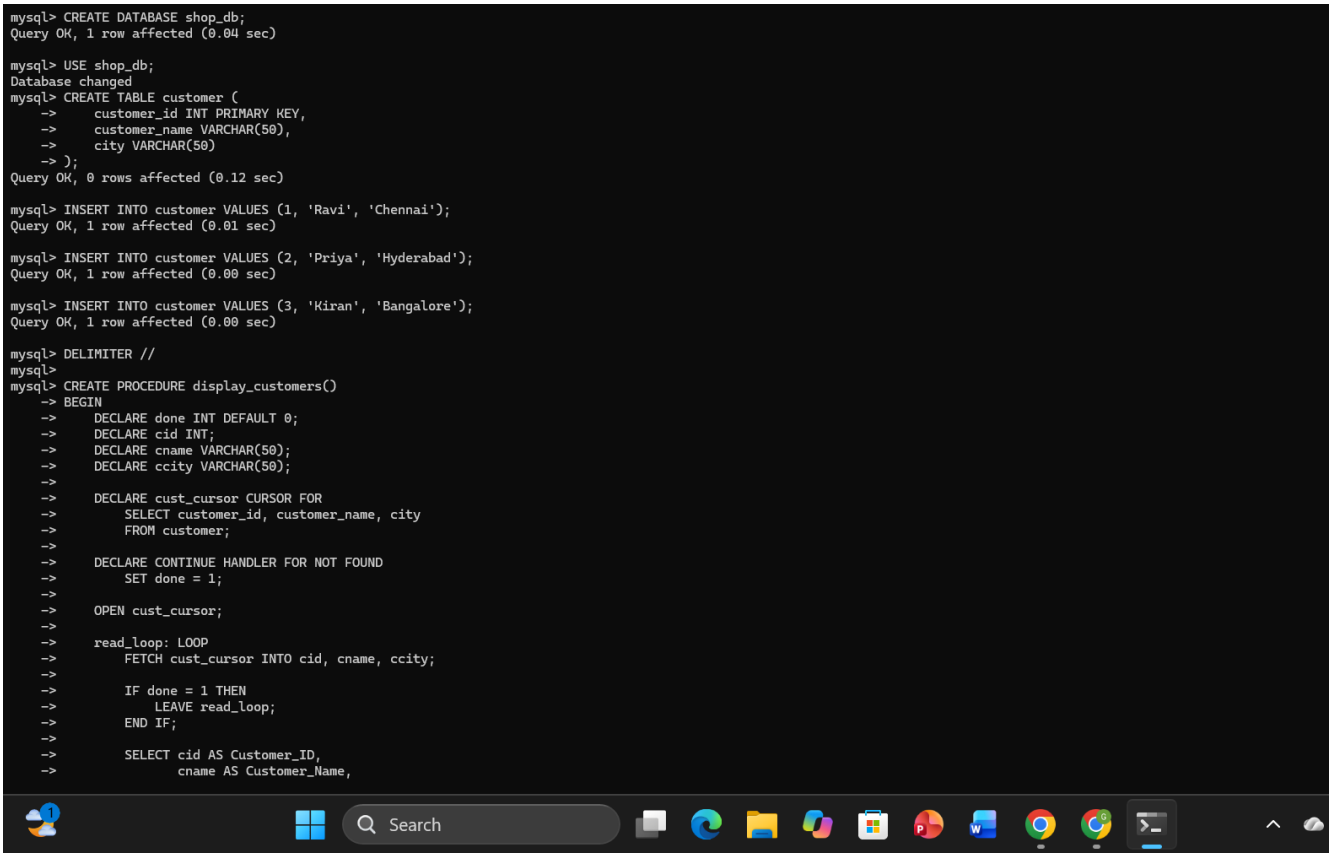
   OPEN emp_cursor;

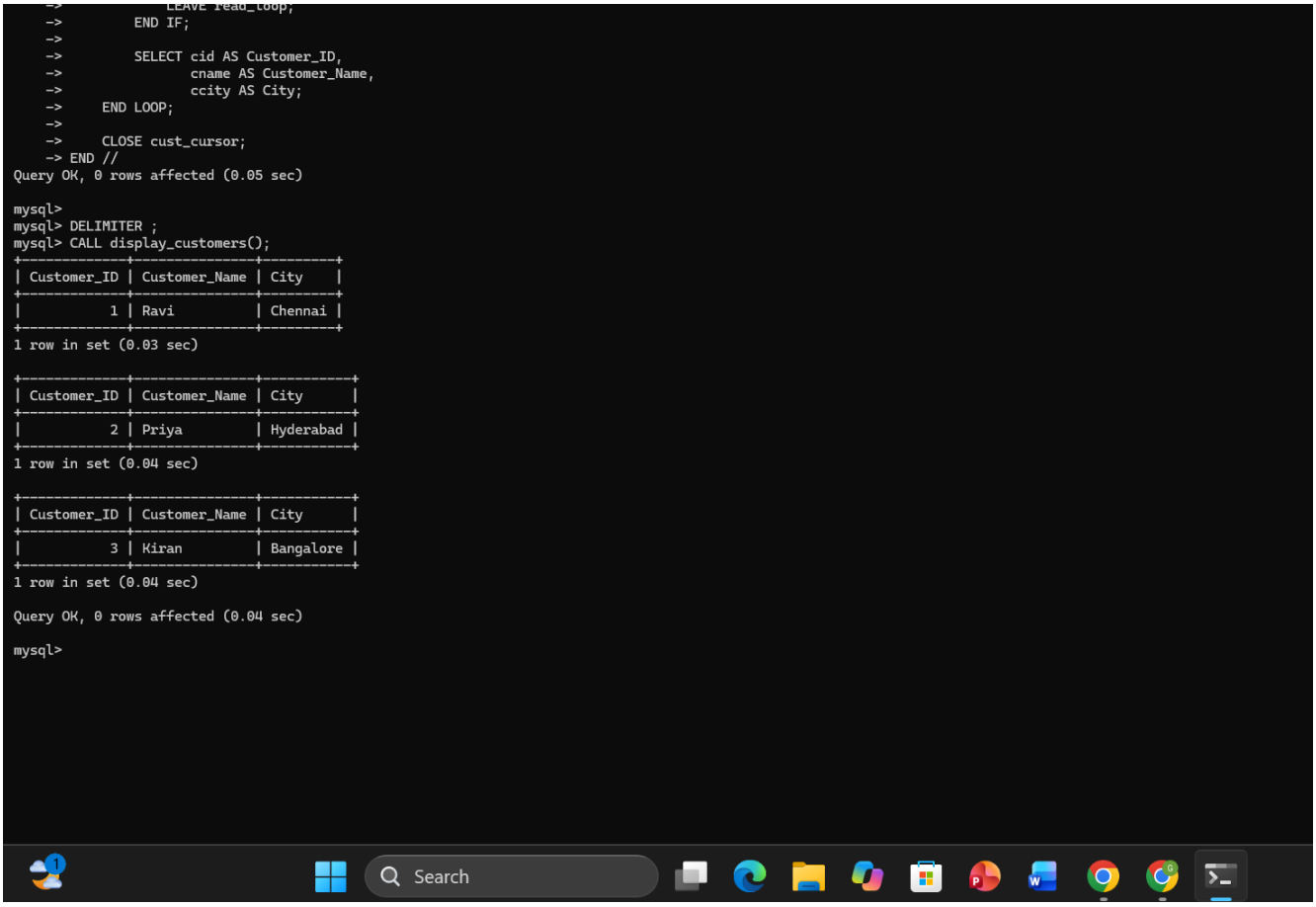
   read_loop: LOOP
      FETCH emp_cursor INTO employee_name;
      IF finished = 1 THEN
         LEAVE read_loop;
      END IF;
      SELECT employee_name AS Employee_Name;
   END LOOP;
```

```
    CLOSE emp_cursor;
END //
DELIMITER ;

CALL DisplayEmployeeNames();
```

## Expected Output:






## Observation

The cursor processes the query result one row at a time. The NOT FOUND handler ends the loop after all employee names have been fetched.

## Result

Thus, a MySQL cursor was declared, opened, fetched, and closed successfully within a stored procedure.